

How Our Pipeline Works

Splitting a 2,000-page mortgage file into clean, structured documents — explained in plain English.

TEAM NOOBDA · INFRX 2026 HACKATHON · PROBLEM B

THE PROBLEM

One blob in. Dozens of documents out.

A mortgage loan file arrives as a single PDF — sometimes 200 pages, sometimes 2,000. Inside it are dozens of completely different documents: bank statements, tax returns, paystubs, credit reports, insurance declarations, purchase contracts — all merged into one continuous blob with no markers, no table of contents, no separators.

The job is to look at that blob and answer two questions:

1 Where does each document start and end?

And if there are 3 tax returns back-to-back, find all 3 — not just one.

2 What are the tables inside?

A bank statement's transaction table often runs across 4–5 pages — stitch those rows back into one.

THE PIPELINE

Five stages, raw PDF to clean JSON

Each stage does one job and hands off to the next. The first three figure out *what* and *where*; the last two reassemble and package.



STAGE 1 – EXTRACT

Get text and tables out of every page

Reads every page and pulls out its text plus any table structure. How it does that depends on whether the page is digital or scanned.

DIGITAL PAGE

Has a real text layer — `pdfplumber` reads text and tables directly. Free and instant.

SCANNED PAGE

- **Tesseract OCR** (free, local) reads the rendered image and returns text.
- **GPT-4o-mini VLM** steps in only when Tesseract returns very little — it sees the actual image and returns text *and* structured tables (headers, rows, columns).

TWO MODES TO CHOOSE FROM

`--no-vlm`

Tesseract only. **~40s** / 125 pages, free, less accurate.

`default`

VLM for scanned pages. **~3 min** / 125 pages, ~\$0.005, **~91% accurate**.

STAGE 2 – CLASSIFY

Label every page with a document type

Every page gets a type — "this is a bank statement," "this is a paystub." There are **27 document types** in all:

bank_stmt_checking

form_1040

paystub

w2

credit_report

closing_disclosure

urla_1003

purchase_contract

+ 19 more

A three-tier cascade — cheapest first

Each page falls through three checks. Most pages settle in the first two for free; only the genuinely ambiguous ones reach the model.

1

Carry-forward

FREE

Is this clearly a continuation of the previous page? Page 3 of a bank statement looks like page 2 — same type, no new header → reuse the previous page's type.

2

Keyword heuristic

FREE

Does the text strongly match a known pattern? `FORM W-2 WAGE AND TAX STATEMENT` → `w2` at confidence 0.95. Returns immediately if confidence ≥ 0.65 .

3

LLM fallback

~\$0.0003 / page

Ambiguous page → GPT-4o-mini with a structured prompt. Only **~19% of pages** actually reach this tier.

Every page leaves Stage 2 with a `doc_type` , a `confidence` , and the `method` that decided it.

STAGE 3 – SEGMENT

Find where each document starts and ends

Walks the pages in order and asks at every gap: *does page B start a new document?* No probabilities here — six hard rules. Any one firing means a boundary.

RULE	WHAT IT CATCHES	EXAMPLE
<code>doc_type_changed</code>	Page B is a completely different type	W2 → Bank Statement
<code>attr_changed</code>	Same type, different instance	Account <code>...1234</code> → <code>...5678</code>
<code>known_length_hit</code>	Doc has a fixed page count	W2 = always 1 page → next is new
<code>balance_break</code>	Ending balance $\neq$ next page's opening	Ends \$5,200 but next opens \$12,000
<code>new_doc_header</code>	Different institution name at top	"Chase" → "Bank of America"
<code>chapter_break</code>	New chapter / story heading	"Chapter 3"

THE TRICK FOR SAME-TYPE DOCS

The `distinguishing_attr` field. Three Form 1040s back-to-back each carry a different tax year — `2022` , `2023` , `2024` . When the year changes, `attr_changed` fires and a new instance begins. Same logic for account numbers on bank statements.

OUTPUT – A LIST OF DOCUMENT INSTANCES

bank_stmt_checking	p18–p38	[CHASE account ending 1234, Jan 2024]
bank_stmt_checking	p39–p51	[CHASE account ending 1234, Feb 2024]
form_1040	p52–p53	[year 2022]
form_1040	p54–p55	[year 2023]

STAGE 4 – STITCH

Reconnect tables split across pages

A bank statement's transaction table gets cut at every page boundary. Stage 4 reconnects those fragments into one logical table.

THE ONLY PROBABILISTIC PART

Two adjacent fragments might belong to the same table or to different ones — hard rules can't always tell. So we use **Naive Bayes** to fuse five signals into one probability.

SIGNAL	WHAT IT CHECKS	WEIGHT
spatial	Do column X-positions line up across the break?	1.8
subtotal	Did the last page end with a TOTAL row? (table ended)	1.5 –
header	Are the column headers the same?	1.2
fingerprint	Do column widths match?	1.1
value_type	Do column data types match (dates, \$, text)?	1.0

THE MATH

```
log_odds      = log(prior) + Σ weight × log( P(signal|same) / P(signal|different) )
P(same_table) = sigmoid(log_odds)

P ≥ 0.75  →  merge   (same table – stitch the rows)
P < 0.30  →  reject  (different tables)
```

STAGE 5 – RENDER

Package it into clean, structured JSON

Everything is assembled into a single JSON output that mirrors the `labels.json` ground-truth format.

```
{
  "documents": [
    {
      "doc_type": "bank_stmt_checking",
      "start_page": 18,
      "end_page": 38,
      "page_count": 21,
      "distinguishing_attr": "CHASE_account_1234_2024-01",
      "tables": [
        {
          "table_id": "table_0040",
          "pages": "18-22",
          "row_count": 87,
          "headers": ["Date", "Description", "Amount", "Balance"]
        }
      ]
    }
  ]
}
```

WHY IT'S EFFICIENT

AI only where it earns its cost

Every stage tries the free path first. The model is a fallback, not a default — so most of the work costs nothing.

STAGE	AI USED?	COST
Digital page extraction	No — pdfplumber	Free
Scanned page OCR	Tesseract first, VLM if needed	Free → ~\$0.002/pg
Classification	Heuristic first, LLM for ~19%	~\$0.0001/pg avg
Segmentation	No — deterministic rules	Free
Table scoring (PTT)	No — math formula	Free
LLM arbiter (stitching)	0% used in practice	Free
Total — 125-page PDF		~\$0.005

At 2,000-page scale with smart routing: ~\$0.08 per file, ~15–20 min end-to-end.

RESULTS

Tested on pkg_000003

172 pages, a mix of digital and scanned documents.

91.9%

Page-level accuracy

24/32

Correct doc instances
(75%)

~\$0.005

Total pipeline cost

~57s

Time (Tesseract) ·
~210s VLM

100%

bank_stmt_checking

100%

paystub

100%

urla_1003

HOW TO RUN

Commands

```
# Full pipeline on one PDF (VLM mode – accurate)
./run.sh pipeline pkg_000003

# Batch all PDFs in DataSet/ – VLM mode
./run_dataset.sh

# Batch all PDFs – Tesseract mode (fast, free)
./run_dataset.sh --fast

# Single PDF directly
python3 src/pipeline/run_pipeline.py --pdf DataSet/doc_003.pdf

# Single PDF, Tesseract only
python3 src/pipeline/run_pipeline.py --pdf DataSet/doc_003.pdf --no-vlm
```

Outputs go to `Predictions/doc_XXX_labels.json`.

FILE STRUCTURE

```
src/  
├─ extraction/      extract.py      - pdfplumber + Tesseract + VLM  
├─ classification/ classify.py      - 3-tier cascade classifier  
├─ segmentation/   segment.py      - 6-signal boundary detection  
│                  reorder.py      - page reordering (shuffled PDFs)  
├─ stitching/      stitch.py       - Naive Bayes table stitcher (PTT)  
└─ pipeline/       run_pipeline.py - 5-stage orchestrator  
  
web/               - Next.js frontend (drag-drop UI)  
DataSet/           - Input PDFs  
Predictions/       - Output labels.json per PDF  
run.sh             - Single package runner  
run_dataset.sh     - Batch runner (all PDFs)
```